Evan Magee

JP McLaughlin

CMPSC 472

Project 2

1. Project Description

The Emergency Room Kiosk Check-In and Scheduling Simulation is designed to model the flow of patients entering an emergency department. It shows the constraints of limited waiting room and exam room capacity, and the impact of different scheduling policies on overall performance. The project integrates a multi-threaded backend with a web-based dashboard that demonstrates how concurrency, synchronization, and queue management work together in a simulated real-life healthcare environment. By simulating multiple check-in kiosks, a triage-based waiting queue, and a centralized scheduler that assigns patients to exam rooms, the system tries to provide a realistic representation of ER workflow under what can be an unpredictable load.

The backend is implemented in C using POSIX threads, semaphores, mutexes, and condition variables to coordinate patient creation and throughput. Four kiosk threads generate new patients at randomized intervals and put them into a bounded waiting room queue. When the queue becomes full, kiosks automatically block until the scheduler frees space which can reflect real constraints on intake capacity. A central scheduler thread monitors patient arrivals, selects which patient should be seen next according to the selected scheduling policy, tracks the length of stay in exam rooms, and computes accurate wait times for every patient who reaches treatment. Throughout execution, the

backend writes structured JSON files that describe the state of the system along with a continuously appended event log.

The frontend uses these files to produce a live dashboard that enables users to observe how the system evolves second by second. It displays kiosk activity, the current scheduling policy, queue length, room occupancy, individual patient details, and average wait times formatted for readability. The dashboard provides insight into throughput and bottlenecks without requiring additional backend computation by reconstructing kiosk statistics directly from patient data. The simulation attempts to highlight the challenges and trade-offs in healthcare scheduling, particularly the balance between patient urgency and resource limitations. This project demonstrates practical applications of operating systems concepts while creating a visualization of ER performance under varying conditions.

2. Project Significance

The significance of this project is that it translates operating systems concepts into a attempt at a realistic simulation of a critical healthcare workflow. Emergency departments are systems full of uncertainty, constrained resources, and the need for rapid decision making. The project shows how low-level synchronization primitives can directly influence high-level operational outcomes by modeling these conditions through a multi-threaded backend and a continuously updating web dashboard. The meaningfulness of the system is in its attempt to show real ER challenges: limited waiting room capacity, competition for exam rooms, triage-based priority decisions, unpredictable arrival patterns, and the cascading effects of bottlenecks. In the simulation you can see how

queue lengths fluctuate, how average wait time evolves, and how different scheduling strategies impact patient flow.

The novelty of the project comes from its integration of systems programming, concurrency control, real-time data reporting, and user-facing visualization into a single cohesive application. This project grounds the simulation in actual thread execution, shared memory coordination, and semaphore-based resource management. This technical foundation makes it so that every patient's movement through the ER is a result of thread interactions and timing conditions. The frontend component adds to the uniqueness of the project by converting raw system behavior into a live, human-readable dashboard.

3. Code Structure

The code structure consists of several backend and frontend files.

- Backend
    - Output
        - appointments.json
            - This file contains arrays of patients waiting and currently being seen. Each patient has an id, name, triage number, kiosk id, status, arrival time, and wait time.
        - log.txt
            - Contains a log of all system operations in the ER system. This includes things like Kiosks adding patients and the Scheduler moving patients into their set exam rooms.
        - status.json
            - This file contains information on the current scheduling priority, total patients, patients with recorded wait, and the last update.
    - Source
        - global.h

        - Defines all shared global settings and variables used by the ER simulation
    - kiosk
        - This file contains the functions for a check-in kiosk in the emergency room. It creates patients in the simulation and inserts them into the waiting queue. If the waiting queue is full or the simulation ends, it will wait for a free slot to open. t
    - queue
        - This file implements a fixed-size queue for storing patients. It provides basic functions such enqueuing, dequeuing, and removing patients at certain positions.
    - scheduler
        - This file is the heart of the ER simulation and is responsible for the movement of patients and the way they are moved. It can move patients based on certain scheduling protocols such as FCFS, Priority, Round Robin, and MLFQ aging. It can also push patients that are at high risk or triage 5 instantly to an exam room. It also records all scheduling activity to the log and creates the Json files to be used by the front end. Finally, it ensures the kiosk threads and scheduler threads coordinate properly.
    - main
        - This file controls the entire ER simulation, initializing the queues, mutexes, and semaphore, and lets the user choose the scheduling policy the simulation will use.
  - makefile
    - This file tells *make* to compile the simulation program generating the o object files and links them together into a single executable. It also allows for *make clean* to be run to clean the compiled program and object files.
- Frontend
  - index.html
    - Builds and contains the dashboard webpage for the ER simulation
  - Script.js
    - This script updates the ER simulation and loads the Json files created by the backend C files connecting them to the frontend. It then

updates the webpage elements to show the changes that are occurring to the user.

- Styles.css
  - Consists of the styling settings for the webpage dashboard

4. Algorithm Description

The project has scheduling algorithms found in operating systems that each illustrate different strategies for prioritizing work in a resource-limited environment. The simplest of these is the First-Come, First-Served (FCFS) algorithm, in which patients are treated strictly in the order they arrive, but since this is supposed to be an emergency room there is an added feature of a level 5(highest priority) acting as an emergency interrupt and being put in a room right away. FCFS emphasizes fairness and transparency but in this case, it also provides room for the highest emergencies since in real-life those situations would not be ignored. Then there is the priority scheduling algorithm, which selects patients based on triage level rather than arrival time. So, patients with more severe conditions are assigned to exam rooms sooner, allowing the simulation to demonstrate the trade-off between fairness and urgency-driven scheduling.

The system also implements Round Robin (RR) which is a time-sliced algorithm. In the context of this project, Round Robin models an ER in which each waiting patient receives a small “time quantum” of attention during the scheduler’s selection process. Round Robin oversees the order in which patients are chosen when multiple candidates are available. This approach prevents any one patient from being continually delayed and gives a more balanced exposure compared to FCFS or priority-only mechanisms.

The last scheduling algorithm in the project is the Multilevel Feedback Queue (MLFQ) with aging, which combines elements of fairness, responsiveness, and starvation prevention. Patients enter the system in a high-priority queue, and their priority may decrease as they wait, causing them to move into lower-level queues over time. However, through aging, patients who have waited for a long time are gradually promoted back into higher-priority queues, ensuring that no individual is neglected indefinitely. We found this algorithm to try to find examples of how a potential real system can work.

5. Algorithm Verification

To verify the correctness of the algorithms and execution of the ER simulation. Each scheduling process was tested under the controlled inputs of the simulation. Testing the FCFS scheduling protocol, the program was shown correctly by selecting the patients by their arrival time. Priority Scheduling also functioned correctly, choosing patients with highest triage value first and preserving the order for those with matching priority. Round Robin correctly operates by cycling through all waiting patients in a fixed order showing no sign of starvation. Finally, MLFQ worked correctly, promoting long-waiting patients' triage levels as they are waiting, preventing starvation, and creating a dynamic priority system.

Kiosk and Schedular thread synchronization was validated using mutexes and conditional variables and showed no signs of race conditions or deadlocks occurring. Finally, the exam-room semaphore reliably limited concurrent patients to rooms, and the JSON and log files matched the patient's flows and simulation operations. Overall, all algorithms in this program functioned as intended.

6. Functionalities

The system provides a complete simulation of an emergency room's intake and scheduling workflow, supported by both backend logic and a real-time frontend user interface.

The first functionality is the patient check-in via multiple kiosks where the system simulates four independent kiosks, each running in its own thread. These kiosks periodically generate new patients with assigned triage levels and arrival times. If the waiting room is full, kiosks temporarily stop accepting new patients until space becomes available. This models the real limitation of intake capacity in a busy emergency department.

The next functionality is a bounded waiting room queue where all incoming patients enter a shared waiting room represented as a queue with a fixed maximum size. Once the queue reaches capacity, no new patients can be added. This ensures that the simulation reflects realistic congestion and forces kiosks to pause when the ER is overloaded.

Then there is the scheduling and patient assignment to exam rooms that features a scheduler thread continuously observing the state of the waiting room and assigning patients to available exam rooms. The selection of the next patient depends on the active scheduling algorithm, which can be FCFS, Priority, Round Robin, or MLFQ with aging. This reproduces different prioritization strategies used in operating systems and healthcare settings.

Next, there is the semaphore-controlled exam room capacity where exam rooms are modeled using a semaphore that tracks how many rooms are currently occupied. The scheduler may only admit a new patient if a room is available, ensuring that treatment capacity is strictly enforced and preventing resource overuse.

Patient wait time measurement is also a functionality where each patient's arrival time is recorded, and their wait time is computed precisely when they first enter an exam room. The system keeps a running total and average wait time across all patients who have begun treatment, allowing clear evaluation of scheduling performance.

Automated JSON and log file generation are also featured where the backend periodically produces updated JSON files listing waiting patients, those being seen, and overall system statistics. A separate log file records key events such as kiosk activity, scheduling decisions, and patient discharges. These files provide a real-time data stream for the frontend.

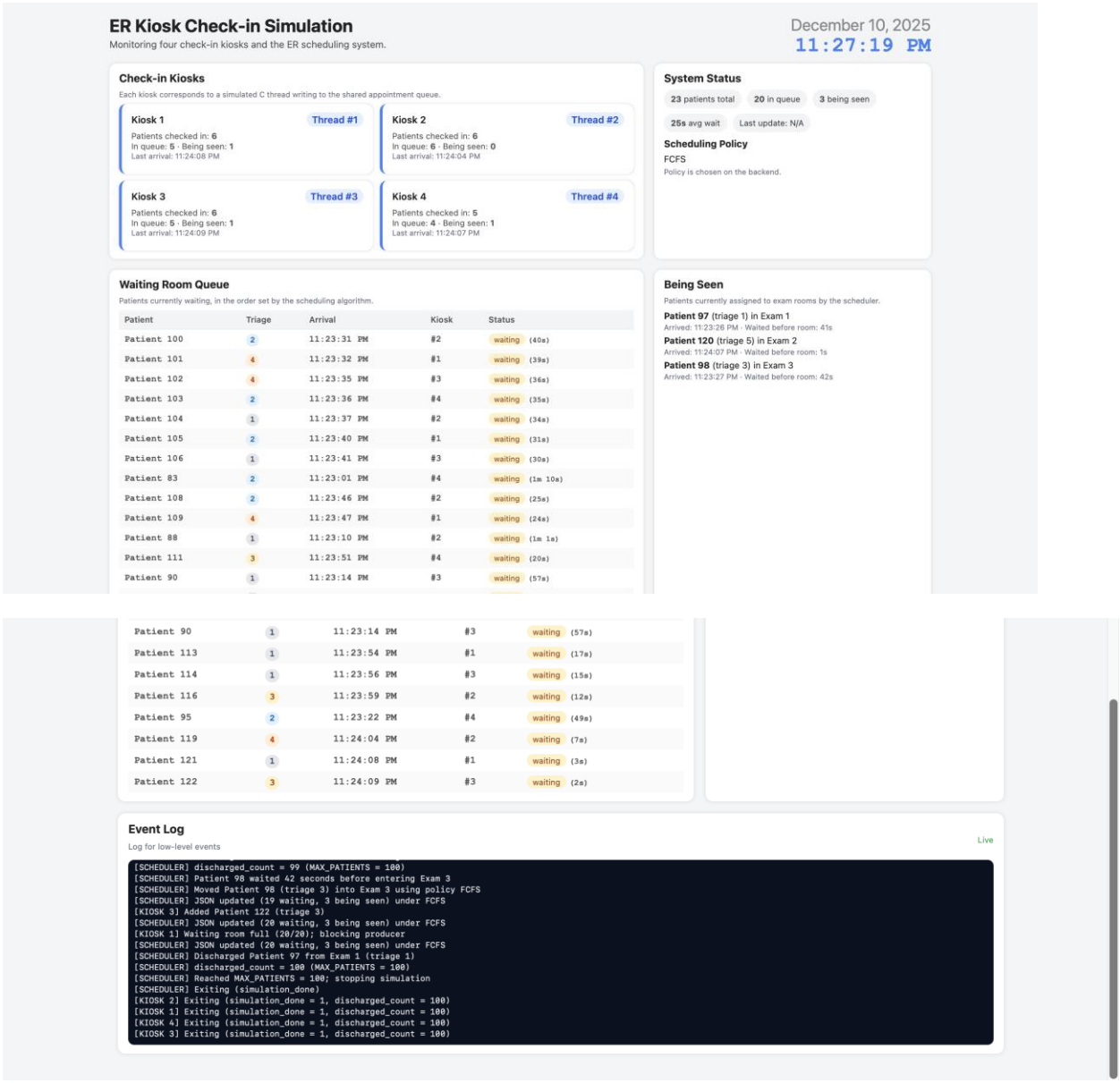
Finally, there is the dynamic, live-updating dashboard where the web-based dashboard retrieves simulation data every second and presents it through tables, status panels, and activity cards. Users can observe the waiting room population, exam room assignments, average wait times, kiosk activity, and the scheduler's real-time behavior.

7. Execution Results and Analysis

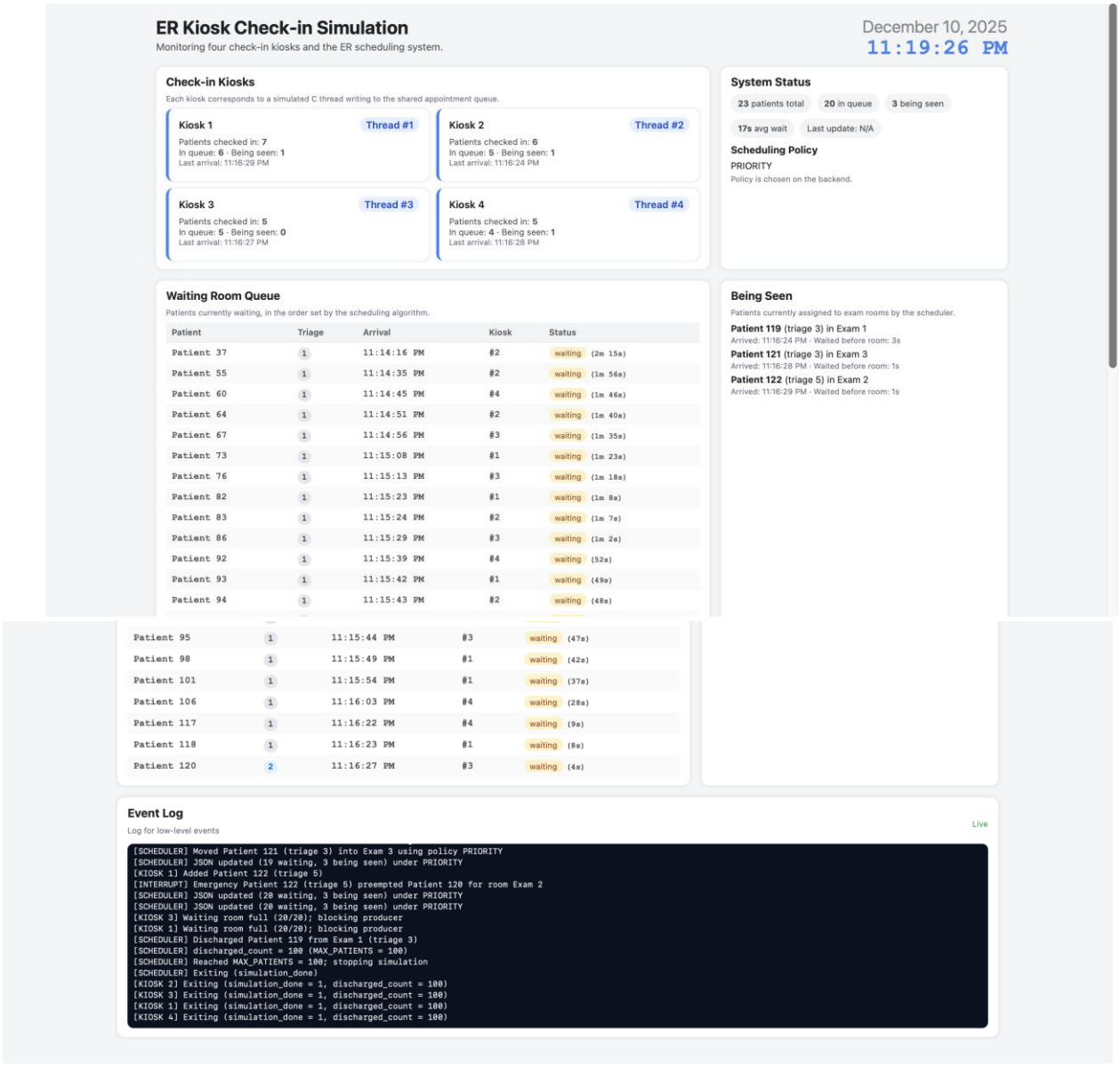
The simulation successfully demonstrated the behavior of different scheduling algorithms under the idea of simulating an emergency room.

- FCFS

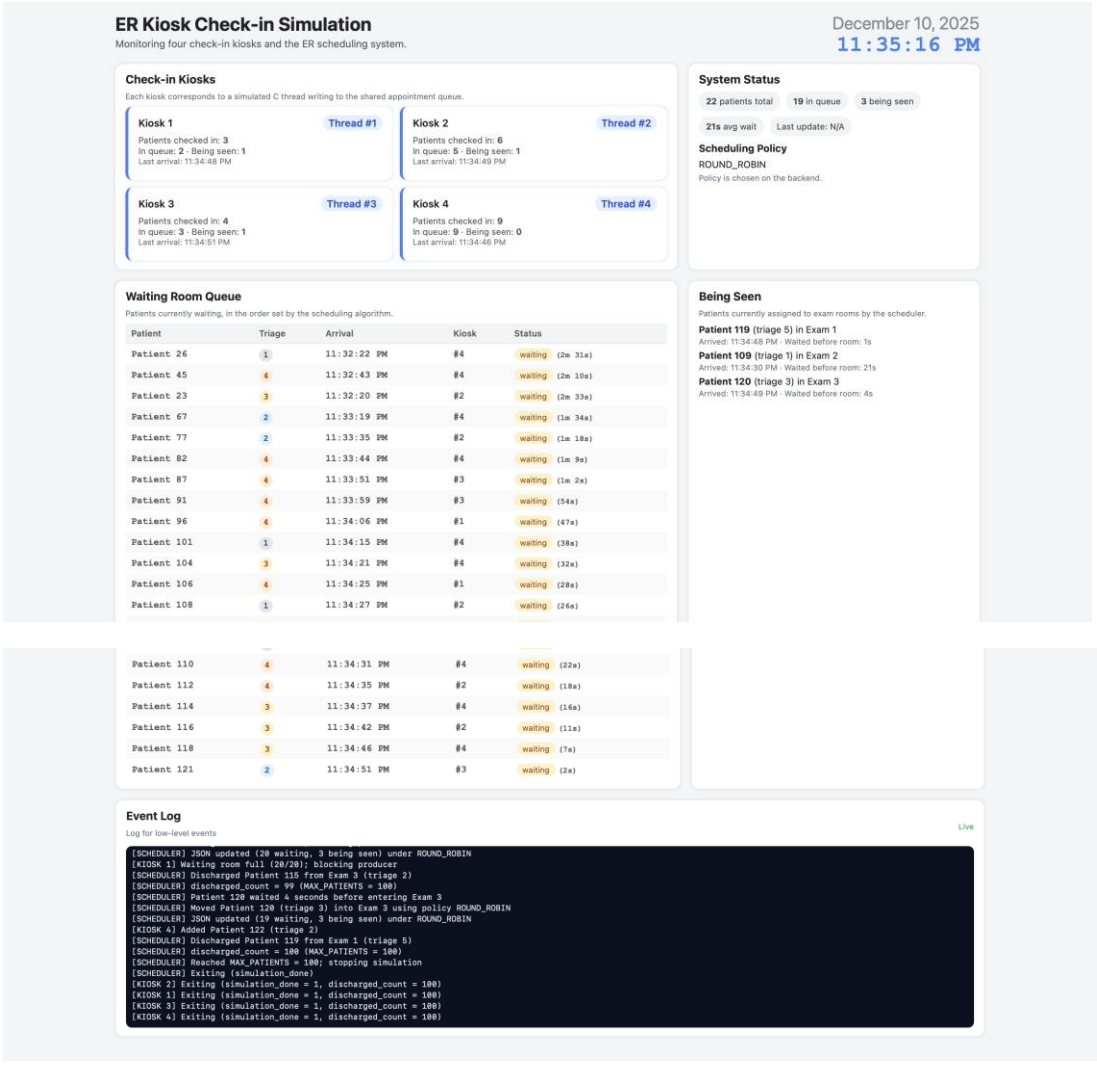
- Processed patients by arrival order. Average Wait = 25s



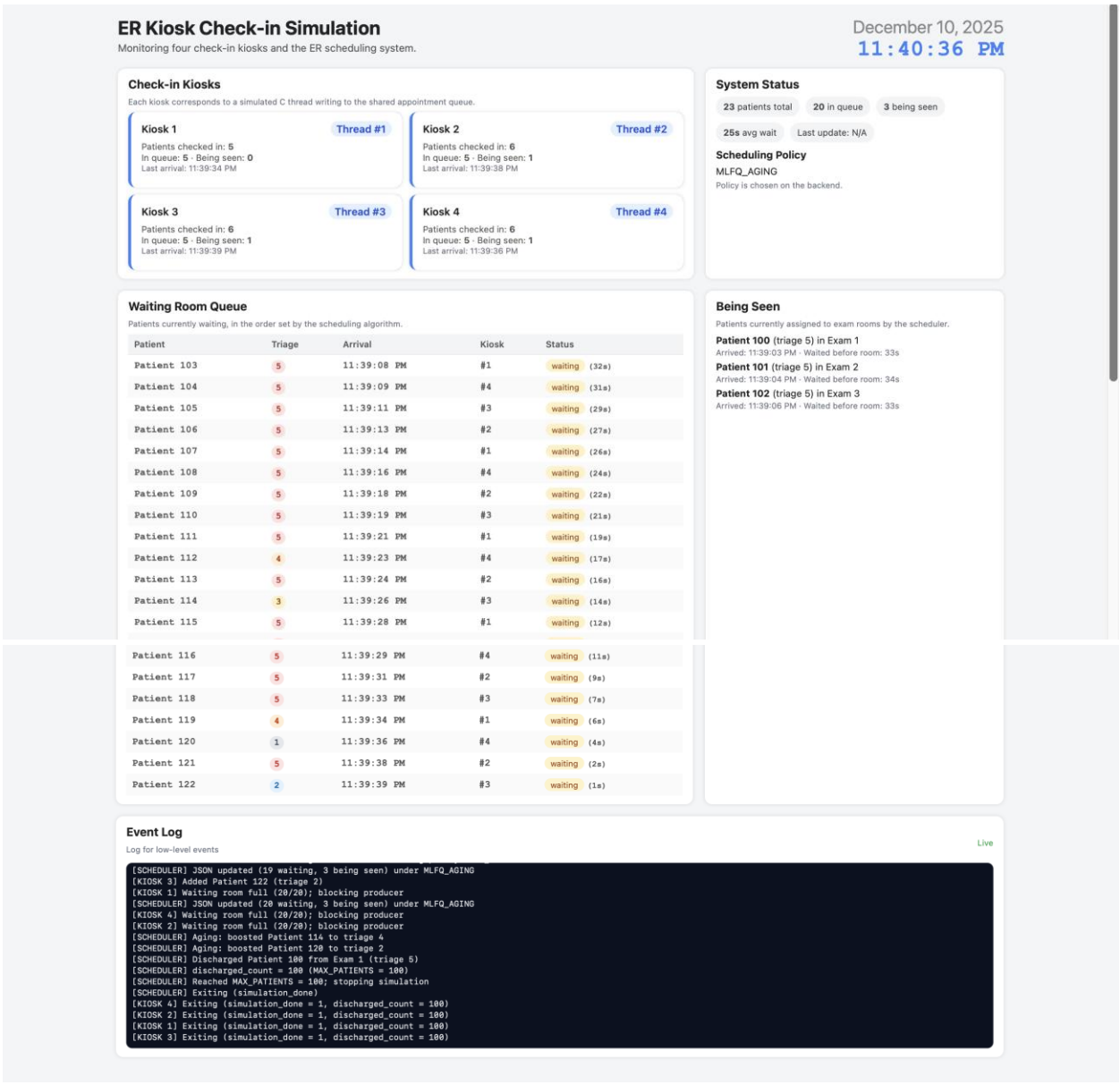
- Priority
  - Processed patients based on triage levels. Average Wait = 27s



- Round Robin
    - Processed patients by cycling through them. Average Wait = 21s



- MLFQ Aging.
    - Processed patients based on wait time and triage level. Average Wait = 25s



Based on the 4 scheduling priorities, Round Robin provided the lowest average wait time. This isn't to mean that it is the best for an ER situation but taking into account just the wait time it is. These results are also in seconds for the sake of the simulation, but realistically, there are many factors in an emergency room that can change how each of the models react or what is the most desired scheduling protocol for an emergency room.

8. Conclusions

The ER Kiosk Simulation successfully demonstrated how operating systems concepts like concurrency, synchronization, and scheduling algorithms can be applied

into a real-world healthcare environment. The implementation of a front-end on top of it allows for clear workers in the ER to gain insights on patient flow, wait times, and resource utilization while trying different scheduling policies. The project also highlights the trade-offs between treating patients based on different metrics such as fairness, urgency, and efficiency. This program also leaves room for modifications on adding more metrics in a real-world setting rather than simulation through the incorporation of metrics like success rate or patient satisfaction. Overall, it validates that these operating system concepts can help find ways to improve real world scenarios through simulations or by interpreting its concepts into different scenarios outside of a computer or code.